

Project 1 COS125 Fall 2025

Recursive Telemetry Analysis with Nested Data

I. General Rules & Academic Integrity

- **Topics:** Practicing elements and concepts covered in class, including; Data type conversion, manipulating lists/strings, complex data types (nested lists and dictionaries), implementing decision/control structures (if/else), functions, and File I/O.
- **Concepts Used:** Only use concepts presented in the course to-date. Using advanced concepts or concepts from outside the course does not demonstrate understanding of the required material and may result in a zero for the assignment.
- **Restrictions:** Do not use explicit loops (for, while), explicit list-manipulation methods that simulate loops (sum(), max() on lists), list comprehensions, breaks, or global values (unless working with shallow copy).
 - All iterative processes MUST be implemented via recursion, including nested recursion to navigate the complex data structure.
- **Allowed Modules:** Only the built-in functions, functions you construct, and the random module is permitted if needed.
- **Academic Integrity:** *Do not use AI.* Asking AI, online searches, or people not associated with the course is not allowed and leads to a grade of zero and an integrity report. Specify any collaboration you had within the comment header. If you went to Boardman 138, state that.
- **Planning:** Start early. Create a written plan of the processes, data flow, logic, and inputs/outputs required, paying special attention to the nested recursion structure.

II. Submission Requirements

- **Planning:** You will submit your planning document to Brightspace prior to creating your program.
- **File Name:** Your Python file must be named using your identity: yourName_NestedTelemetry.py (e.g., lauraGurney_NestedTelemetry.py).
- **Comment Header:** Include the required comment header in your file and ensure all data within the header is updated.
- **Folder Name:** Place the .py file into a folder named yourName_NestedTelemetry.
- **Submission:** Compress the folder (.zip) and submit the compressed file containing all project files.
- **INCLUSION OF FILES not related to the assignment will result in grade reduction of 50%.**

III. Objective & Data Structure

Objective

Demonstrate implementation of a modular Python program that processes a large, hierarchically structured data file using nested recursion. The program must navigate three levels of data structure to find and report specific maximum and minimum values.

Data File Structure

The data file used for grading is `combined_nested_telemetry.txt`, which contains 550 total records.

Data Format. You will be effectively building a master list containing dictionaries for each segment ID, which have the subsystems as keys and lists of the inside data for values.

The file is structured into three nested levels:

1. Top Level:

Mission Segment (Container)

- **Marker in File:** Headers like `# --- SEGMENT START: M1...` and identifiers such as `[SEGMENT_ID:T1]`.
- **Data Structure Role:** This forms the outer list in your final Python structure, where each item in the list is a dictionary representing an entire mission segment.

2. Middle Level: Subsystem Group (Container)

- **Marker in File:** Headers like `[SUBSYSTEM:P]` (Power), `[SUBSYSTEM:T]` (Thermal), and `[SUBSYSTEM:G]` (Guidance).
- **Data Structure Role:** This is represented by keys within the segment dictionary (e.g., `"pulses_P"`), which point to a list of the actual telemetry readings.

3. Bottom Level: Telemetry Pulse (Leaf Node)

- **Marker in File:** A single line of comma-separated values, such as `1000,95.50`.
- **Data Structure Role:** This is a flat list of dictionaries where each pulse is a simple dictionary containing `{'time': 1000, 'value': 95.50}`. Flat list means not nested.

4. Pulse Data Breakdown:

The value to the **left of the comma** is the **Timestamp** (Independent Variable).

The value to the **right of the comma** is the **Metric Value** (Dependent Variable), which is analyzed for Max/Min.

IV. Initial Setup and Requirements

- User Input and Initialization

The program must begin by prompting the user for the following inputs:

- **Subsystem Target:** Enter a single letter (e.g., "P", "T", "G") to specify the subsystem to analyze.
- **File Name:** Use the telemetry file provided.

- Data Analysis Targets

The program must calculate four required maximum and minimum values for the user's chosen subsystem:

- **Overall Maximum:** The highest metric value found across all 550 records (Segments T1, T2, M1, M2, M3, M4, M5).
- **Overall Minimum:** The lowest metric value found across all 550 records.
- **Thermal Maximum:** The highest metric value found within the records (Segments T1 and T2).
- **Thermal Minimum:** The lowest metric value found only within the records (Segments T1 and T2).

V. Recursive Function Implementation

Implement functions using **recursion only** (no loops).

Task 1: Recursive Data Parsing

- **Function:** function to recursively process data passed in by the function call.
- **Input:** A list of raw string lines read from the input file and passed in by the function call.
- **Output:** A complex, nested list-of-dictionaries.
- **Logic:** The function must recursively process the list of lines, identifying segment and subsystem markers to build the required nested list of dictionaries.

Task 2: Recursive Max/Min Calculation (Nested Recursion)

- **Function:** Recursive analysis of the nested list of dictionaries for subsystem data passed in by the function call and the user indicated target subsystem to search for, also passed in by the function call.
- **Input:** The nested list of dictionaries (output of Task 1) and the user's target subsystem string (e.g., 'P').
- **Output:** A tuple or list containing two numeric values for the overall maximum the overall minimum found.
- **Logic:** This function requires recursion, may need nested recursion.
 - The outer recursive call iterates through the list of Mission Segment dictionaries.

- The inner recursive call (or helper function) descends into the subsystem pulse list (e.g., pulses_P) to find the local maximum and minimum for that segment.
- The outer call then recursively compares the local max/min values to determine and return the final, overall maximum and minimum.
- **Alternative:** *You may implement separate recursive functions for maximum and minimum. Please make a note for which version you opt to use.*

Task 3: Recursive Segment Filtering & Analysis

- **Function:** Function to recursively filter the segments from the nested list of dictionaries by the segment ids (T1, T2, etc.) as passed in from the function call
- **Input:** The full nested list of dictionaries and a list of target segment IDs (e.g., ['T1', 'T2']). Design so that the list of elements can be from a minimum of 1 list item to many.
- **Output:** A new nested list containing only the data for the specified segments.
- **Logic:** Recursively filters the top layer of the data structure. The output of this filtered data must be passed to your Task 2 function to calculate the thermal Maximum and thermal Minimum. The T designator stands for Thermal.

VI. Standard Simulation and Logging

Generate a final summary report.

- Summary report will contain all required results and write it to the *analysis_log.txt* file in write only format. May hard code the name of the log file to start with.
- Set up to create a file that does not exist, if it does already exist use error handling to ask for a new name for the log.
- Implement error catching clearly and thoughtfully to protect existing files.

The log file must clearly and readably present the following seven pieces of information:

- **Telemetry File Processed:** The file name.
- **Target Subsystem:** The Subsystem ID entered by the user.
- **Total Pulses Analyzed:** The count of records.
- **Overall Maximum Value:** The maximum value calculated from the full dataset (Output of Task 2).
- **Overall Minimum Value:** The minimum value calculated from the full dataset (Output of Task 2).
- **Segment Maximum:** The maximum value calculated from the filtered T1/T2 data.
- **Segment Minimum:** The minimum value calculated from the filtered T1/T2 data.

See submission guidelines on at the beginning of the instructions.